

Báo cáo chi tiết YC1

Huấn luyện & Đánh giá Mô hình Nhận dạng Người nói (ECAPA-TDNN)

Đề án: Secure Virtual Assistant with Speaker Recognition

Tháng 6, 2026

Phạm vi: Báo cáo tập trung vào Yêu cầu 1 (YC1) — *huấn luyện và đánh giá* mô hình ECAPA-TDNN cho Speaker Identification (SID) và Speaker Verification (SV). Trình bày chi tiết *từng bước* với *trích đoạn code thật* kèm giải thích, sơ đồ, công thức và bộ kết quả thực nghiệm trên hai dataset *VoxCeleb Indian* và *VIVOS*. Không đi sâu vào tầng bảo mật/ứng dụng (YC2) ngoài cách runtime sử dụng SID/SV score. Tài liệu viết bằng tiếng Việt.

Contents

1	Mục tiêu của YC1	3
2	Các file cần đọc trong phần training	3
3	Bức tranh tổng thể: pipeline huấn luyện và suy luận	3
4	Dữ liệu và cách chia tập	4
4.1	Hai bộ dữ liệu	4
4.2	Vì sao chọn hai dataset này?	4
4.3	Quá trình chuẩn bị dataset (5 bước)	5
4.4	Tạo split phân tầng theo từng người nói	5
4.5	Sinh trial pair cho SV	5
5	Tiền xử lý chi tiết	6
5.1	Chuẩn hoá waveform: resample, mono, cắt/đệm 3 giây	6
5.2	Trích đặc trưng Mel-filterbank + log + CMN	6
5.3	Augmentation tùy chọn (đã có khung, mặc định tắt)	7
6	Kiến trúc mô hình và hàm mất mát	7
6.1	ECAPA-TDNN	7
6.2	AAM-Softmax (Additive Angular Margin)	7
7	Vòng lặp huấn luyện	8

8	Fine-tune: huấn luyện tiếp từ checkpoint	8
8.1	Fine-tune ở đâu?	8
8.2	Fine-tune như nào? (cơ chế)	9
8.3	Khi nào nên fine-tune?	9
9	Đánh giá mức cải thiện (before vs after)	9
10	Đánh giá SID và F1-score	10
10.1	Suy luận SID bằng cosine với trọng số lớp	10
10.2	F1-score: micro và macro	11
10.3	Bổ sung tính macro-F1 (đã nâng cấp <code>evaluate_sid.py</code>)	11
11	Đánh giá SV: EER và minDCF	11
11.1	Trích embedding toàn utterance	11
11.2	EER — Equal Error Rate	11
11.3	minDCF — chi phí quyết định tối thiểu	11
11.4	Quyết định pass/block theo ngưỡng	12
12	Kết quả và so sánh	12
12.1	Lịch sử thực nghiệm và độ ổn định (V1/V2/V3)	14
13	Giải thích các file output	15
14	Lý do chọn mô hình và ưu điểm	15
15	Hạn chế của phương pháp	15
16	Hướng cải tiến	16
17	Câu hỏi dự đoán và đáp án	16
18	Checklist ôn tập	17
19	Tóm tắt một đoạn để nói	17
20	Gợi ý thứ tự trình bày 5 phút	18
21	Kết luận	18
22	Phụ lục: lệnh tái lập	18

1 Mục tiêu của YC1

YC1 trả lời hai câu hỏi nền tảng của trợ lý ảo có giọng nói:

- **Ai đang nói? — Speaker Identification (SID):** phân loại giọng nói vào một trong N người đã đăng ký (closed-set, multi-class).
- **Có đúng là người đó không? — Speaker Verification (SV):** quyết định nhị phân chấp nhận/từ chối khi so giọng hiện tại với một người cụ thể.

Tư tưởng cốt lõi

Hệ thống dùng **một mô hình embedding duy nhất** cho cả hai bài toán: huấn luyện theo kiểu *phân loại người nói* (speaker classification), sau đó lấy vector embedding 192 chiều và so khớp bằng **cosine similarity**. SID chọn người có cosine cao nhất; SV so cosine với một ngưỡng. Đây là chuẩn của các hệ thống speaker recognition hiện đại.

2 Các file cần đọc trong phần training

File	Vai trò trong YC1
training/train_ecapa.py	Toàn bộ logic train: Dataset, tiền xử lý, mel, ECAPA-TDNN, AAM-Softmax, vòng lặp train/val, lưu checkpoint.
training/evaluate_sid.py	Đánh giá SID: Top-1/Top-5, và (bản nâng cấp) micro/macro/weighted-F1 + confusion matrix.
training/evaluate_sv.py	Đánh giá SV: EER, minDCF, threshold; (bản nâng cấp) lưu (label, score) để vẽ DET/ROC.
training/augmentations.py	Khung augmentation tùy chọn: speed, MUSAN noise, RIR reverb, SpecAugment.
notebooks/SecureVA-V*.ipynb	Bản vòng thực nghiệm trên Kaggle; V3 sinh evidence chính thức.
results/<dataset>/	Artifact: checkpoint, log, split, kết quả SID/SV, hparams.

Table 1: Bản đồ đọc code cho phần huấn luyện.

3 Bức tranh tổng thể: pipeline huấn luyện và suy luận

Quá trình gồm hai giai đoạn dùng *chung* mạng ECAPA-TDNN.

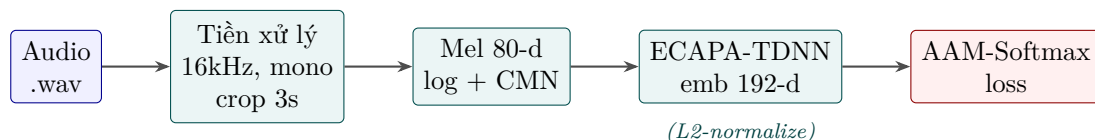


Figure 1: Giai đoạn HUẤN LUYỆN: tối ưu cross-entropy có biên góc trên nhãn người nói.

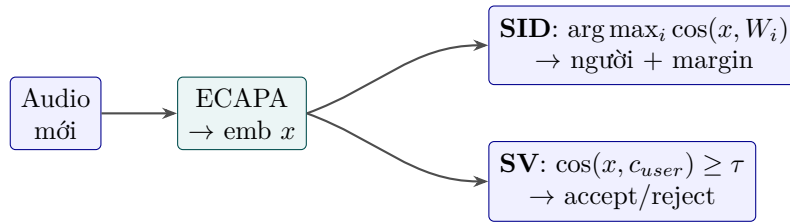


Figure 2: Giai đoạn SUY LUẬN: cùng embedding x phục vụ cả SID (so với trọng số lớp/centroid) và SV (so với centroid của 1 user theo ngưỡng τ).

Câu nên nói khi vấn đáp

Mô hình *không* được train trực tiếp cho SV. Nó được train để *phân loại* người nói; chính quá trình đó ép embedding của cùng một người xích lại gần nhau và khác người thì tách xa — nhờ vậy cosine giữa hai embedding trở thành thước đo “cùng người hay không”.

4 Dữ liệu và cách chia tập

4.1 Hai bộ dữ liệu

Dataset	Spk	Utt	Train	Val	Test	Trial SV
VoxCeleb Indian	24	4,857	3,389	717	751	552
VIVOS	65	12,419	8,686	1,850	1,883	4,160

Table 2: Thống kê (seed = 42). VIVOS gộp 46 speaker tập train + 19 speaker tập test.

4.2 Vì sao chọn hai dataset này?

Việc chọn dataset bám theo *mục tiêu sử dụng* (trợ lý ảo tiếng Việt) và *ràng buộc tài nguyên* (train miễn phí trên Kaggle GPU). Lý do cụ thể:

- **VIVOS — dataset chính:**

- *Đúng miền tiếng Việt:* người dùng thật nói tiếng Việt; model train trên giọng Việt sẽ không bị lệch miền khi deploy.
- *Đủ lớn cho closed-set:* 65 người $\times$ 12k câu — đủ để học embedding phân biệt mà vẫn train được trong vài giờ trên 1 GPU.
- *Sạch, gán nhãn rõ:* là corpus ASR tiếng Việt chuẩn (ĐH KHTN), nhãn speaker tin cậy, license học thuật rõ ràng.
- *Miễn phí, có sẵn trên Kaggle* $\rightarrow$ tái lập dễ.

- **VoxCeleb Indian subset — dataset đối chứng:**

- *Kiểm chứng tính tổng quát:* audio YouTube “in-the-wild” (nhiều, đa micro, đa môi trường) — chứng minh pipeline không chỉ chạy tốt trên dữ liệu sạch.
- *Là chuẩn học thuật:* VoxCeleb là benchmark speaker recognition phổ biến nhất $\rightarrow$ kết quả dễ đối chiếu với văn liệu.
- *Subset Indian nhỏ gọn* (24 người): vừa đủ tài nguyên Kaggle, không cần tải VoxCeleb full (hàng trăm GB).

Tóm tắt lý do chọn dataset

VIVOS để **khớp miễn** (tiếng Việt, sạch, đúng đối tượng người dùng); VoxCeleb Indian để **đối chứng độ bền** trên dữ liệu nhiễu thực tế. Cả hai đều *miễn phí + có trên Kaggle* nên train lại được, và *đủ nhỏ* để vừa tài nguyên GPU free-tier.

4.3 Quá trình chuẩn bị dataset (5 bước)

Toàn bộ khâu chuẩn bị nằm trong notebook `SecureVA-V3.ipynb`, gồm 5 bước:

1. **Tải & gom dữ liệu:** add dataset trên Kaggle; với VIVOS, gộp `train/` và `test/` vào một thư mục `vivos_speaker_root` để có đủ 65 speaker trong một closed-set.
2. **Quét file & loại audio lỗi:** duyệt từng `.wav`, thử `torchaudio.load`; file hỏng bị ghi vào `bad_audio_files.txt` và loại khỏi split (VIVOS: 1 file). Chỉ giữ speaker có ≥ 3 utterance để chia được cả 3 tập.
3. **Chia split phân tầng theo speaker** (70/15/15) — xem Mục 4.4.
4. **Sinh trial pair SV** (cân bằng dương/âm) — xem Mục 4.5.
5. **Dump thống kê:** ghi `dataset_summary.json` (số speaker, utterance, kích thước split, số trial) để kiểm tra và đưa vào báo cáo.

4.4 Tạo split phân tầng theo từng người nói

Mỗi người nói được xáo trộn *riêng* rồi chia 70/15/15, đảm bảo *mọi speaker đều xuất hiện ở cả ba tập* — đúng định nghĩa closed-set SID.

```
for spk, files in spk2files.items():
    random.shuffle(files)
    n = len(files)
    n_train = max(1, int(0.70 * n)); n_val = max(1, int(0.15 * n))
    train, val, test = files[:n_train], files[n_train:n_train+n_val], files[n_train+n_val:]
    for p in train: iden_lines.append(f"1 {p}\n")
    for p in val: iden_lines.append(f"2 {p}\n")
    for p in test: iden_lines.append(f"3 {p}\n")
```

Giải thích: file `iden_split.txt` là “mục lục” của dataset; mỗi dòng gồm mã split (1/2/3) và đường dẫn tương đối. `train_ecapa.py` chỉ đọc các dòng đúng split cần dùng. Cách chia theo speaker (thay vì trộn toàn cục) tránh việc một người vắng mặt khỏi tập train.

4.5 Sinh trial pair cho SV

SV cần các *cặp thử*: nhãn 1 (cùng người) hoặc 0 (khác người), số cặp dương/âm được cân bằng.

```
# Cap duong: ghép các file CÙNG một người (giới hạn 3 cap/file để không bùng nổ)
for files in test_by_spk.values():
    for i in range(len(files)):
        for j in range(i+1, min(i+4, len(files))):
            pos.append((1, files[i], files[j]))
# Cap am: ghép file của HAI người khác nhau
for i in range(len(spks)):
    for j in range(i+1, len(spks)):
        neg.append((0, random.choice(test_by_spk[spks[i]]), random.choice(test_by_spk[spks[j]])))
k = min(len(pos), len(neg)); trials = random.sample(pos, k) + random.sample(neg, k)
```

Giải thích các lựa chọn: (i) *cap 3 cặp dương/file* tránh bùng nổ số cặp khi một người có nhiều câu (tổ hợp C_n^2 rất lớn); (ii) *cân bằng dương/âm* ($k = \min$) để EER không bị lệch về một phía; (iii) chọn 70/15/15 là tỉ lệ phổ biến — đủ dữ liệu train, vẫn còn val để chọn checkpoint và test để báo cáo.

Caveat về dữ liệu — cần nói rõ khi vấn đáp

- (1) **Rò rỉ phiên:** split phân tầng theo speaker nhưng *không* nhóm theo video_id (phiên YouTube) $\Rightarrow$ utterance cùng phiên có thể nằm cả train lẫn test, khiến chỉ số *lạc quan hơn* thực tế. Đây **không phải** bộ VoxCeleb1-O chính thức.
- (2) **Trial nhỏ (VoxCeleb):** 552 cặp $\rightarrow$ EER có sai số thống kê rộng ($\sim \pm 2\%$).
- (3) **Lệch miền:** VoxCeleb Indian là giọng Ấn (in-the-wild, nhiều); VIVOS là giọng đọc sạch — dễ hơn hội thoại thực.

5 Tiền xử lý chi tiết

Đây là phần “làm gì với audio trước khi vào mạng”. Cài đặt ở VoxCelebSID._load và FbankExtractor.

5.1 Chuẩn hoá waveform: resample, mono, cắt/đệm 3 giây

```
wav, sr = torchaudio.load(str(path))
if sr != self.sample_rate:                # (1) đưa về 16 kHz
    wav = torchaudio.functional.resample(wav, sr, self.sample_rate)
wav = wav.mean(dim=0)                      # (2) tron stereo -> mono
if wav.size(0) >= self.num_samples:       # (3) cắt về đúng 3s
    if self.split == "train":
        start = random.randint(0, wav.size(0) - self.num_samples) # random crop
    else:
        start = (wav.size(0) - self.num_samples) // 2              # center crop
    wav = wav[start:start + self.num_samples]
else:
    wav = F.pad(wav, (0, self.num_samples - wav.size(0)))         # zero-pad nếu ngắn
```

Giải thích từng bước:

- Resample 16 kHz:** thống nhất tần số lấy mẫu — ECAPA và mel-filter đều giả định 16 kHz.
- Mono:** lấy trung bình kênh, vì danh tính người nói không phụ thuộc số kênh.
- Cắt 3 giây** (48,000 mẫu): khi train dùng *random crop* — mỗi epoch thấy một đoạn khác nhau, đóng vai trò augmentation thời gian *miễn phí*; khi eval dùng *center crop* để kết quả tái lập. Audio ngắn hơn 3s thì *zero-pad* ở cuối.

Vì sao random vs center crop?

Random crop lúc train giúp mô hình không học thuộc một đoạn cố định, tăng tính tổng quát. Center crop lúc eval khử yếu tố ngẫu nhiên để hai lần chạy ra cùng số. Khi *trích embedding lúc deploy*, hệ thống dùng *toàn bộ utterance* (không crop) để embedding ổn định nhất.

5.2 Trích đặc trưng Mel-filterbank + log + CMN

```

self.mel = torchaudio.transforms.MelSpectrogram(
    sample_rate=16000, n_fft=512, win_length=400, # 25 ms
    hop_length=160, n_mels=80) # 10 ms hop, 80 bins
def forward(self, x):
    m = self.mel(x) # pho Mel
    m = torch.log(m + 1e-6) # nen dai dong (log)
    m = m - m.mean(dim=-1, keepdim=True) # CMN: tru trung binh theo thoi gian
    return m.transpose(1, 2) # -> [B, T, 80] cho ECAPA

```

Giải thích: mỗi 10 ms sinh một khung 80 chiều mô tả năng lượng theo dải tần Mel (gần với cảm nhận tai người). Hai phép hậu xử lý quan trọng:

- **Log:** $\log(\text{mel} + 10^{-6})$ nén dải động (biên độ chênh lệch lớn) về thang dễ học; 10^{-6} tránh log 0.
- **CMN (Cepstral Mean Normalization):** trừ trung bình theo trục thời gian, $m' = m - \overline{m}_t$. Loại thiên lệch do *kênh truyền/micro* — giúp embedding bền hơn khi đổi thiết bị thu.

5.3 Augmentation tùy chọn (đã có khung, mặc định tắt)

`augmentations.py` cung cấp pipeline chỉ chạy khi train. Ví dụ trộn nhiễu MUSAN theo SNR mục tiêu:

```

def _mix_noise(self, wav):
    noise = _fit_length(_load_mono(random.choice(self.musan_files), sr), wav.numel())
    snr = random.uniform(*self.snr_db) # vd 5..20 dB
    scale = torch.sqrt(clean_power / (noise_power * 10 ** (snr/10)))
    return wav + scale * noise # tron theo SNR

```

Ngoài ra có **speed perturb** (0.9/1.0/1.1×), **RIR reverb** (tích chập đáp ứng phòng) và **SpecAugment** (che ngẫu nhiên dải tần/thời gian trên log-mel).

Trạng thái trong evidence

Trong các kết quả của báo cáo này, augmentation **tắt** (`augment=false` trong `hparams.json`) do môi trường Kaggle chưa có dataset MUSAN/RIR hợp lệ. Đây là một hướng cải tiến đã sẵn khung (xem Mục 16).

6 Kiến trúc mô hình và hàm mất mát

6.1 ECAPA-TDNN

```

model = ECAPA_TDNN(input_size=80, lin_neurons=192,
    channels=[512, 512, 512, 512, 1536]) # ~6.2M params

```

ECAPA-TDNN (Desplanques 2020) là baseline state-of-the-art cho speaker embedding: dùng *SE-Res2Block* (chú ý kênh), *multi-layer feature aggregation* và *attentive statistics pooling* để gộp toàn bộ utterance thành một vector 192 chiều. Dùng cài đặt SpeechBrain để khởi “phát minh lại bánh xe”.

6.2 AAM-Softmax (Additive Angular Margin)

```

def forward(self, embeddings, labels):
    emb_n = F.normalize(embeddings, dim=1); w_n = F.normalize(self.weight, dim=1)
    cos_theta = F.linear(emb_n, w_n).clamp(-1+1e-7, 1-1e-7) # cos goc emb->lop
    theta = torch.acos(cos_theta)

```

```
target = F.one_hot(labels, cos_theta.size(1)).bool()
theta_m = torch.where(target, theta + self.margin, theta) # cộng biên vào lớp đúng
logits = self.scale * torch.cos(theta_m) # margin=0.2, scale=30
return F.cross_entropy(logits, labels), acc
```

Công thức. Với embedding chuẩn hoá, logit của lớp đúng y_i bị cộng biên góc m :

$$\mathcal{L} = -\log \frac{e^{s \cdot \cos(\theta_{y_i} + m)}}{e^{s \cdot \cos(\theta_{y_i} + m)} + \sum_{j \neq y_i} e^{s \cdot \cos \theta_j}}, \quad m = 0.2, s = 30.$$

Vì sao AAM-Softmax thay cho Cross-Entropy thường?

Cộng biên m vào góc của lớp đúng buộc mô hình phải kéo embedding gần centroid lớp đúng *chặt hơn* mới giảm được loss — tạo khoảng cách *inter-class* lớn và *intra-class* nhỏ trong không gian cosine. Vì SV/SID quyết định bằng chính cosine, biên góc này trực tiếp cải thiện độ tách biệt giữa các người nói. Hệ số s (scale) phóng đại logit để softmax học ổn định.

7 Vòng lặp huấn luyện

```
emb = model(feat(wav)).squeeze(1) # mel -> embedding
loss, acc = classifier(emb, label) # AAM-Softmax
optimizer.zero_grad(); loss.backward()
torch.nn.utils.clip_grad_norm_(params, 5.0) # chong gradient no
optimizer.step()
# ... cuối mỗi epoch:
scheduler.step() # CosineAnnealingLR
if val_acc > best_val: # lưu checkpoint TOT NHAT theo val
    torch.save({"model":..., "classifier":..., "epoch":epoch, "val_acc":val_acc}, "best_model.pt")
```

Các quyết định huấn luyện:

- **Adam**, $lr = 10^{-3}$, weight decay 2×10^{-5} .
- **CosineAnnealingLR**: giảm dần learning rate theo cosine $\rightarrow$ hội tụ mượt ở cuối.
- **Gradient clipping** max-norm 5.0: chặn bước cập nhật quá lớn (ổn định).
- **Early-best**: chỉ lưu checkpoint khi *val accuracy* cao nhất — nên checkpoint VIVOS là epoch 10 dù train 15 epoch.
- Batch 64, 15 epoch, seed 42 (kèm `hparams.json` ghi Python/PyTorch/git SHA để tái lập).

Phân biệt: train từ đầu vs fine-tune

Evidence chính (V3) **train từ đầu** (`train_ecapa.py`) — ECAPA khởi tạo ngẫu nhiên rồi học trên dataset. *Fine-tune* là kịch bản train *tiếp* từ một checkpoint đã có (Mục 8), dùng khi muốn thêm augmentation, thêm người nói mới, hoặc đẩy thêm độ chính xác mà không train lại từ con số 0.

8 Fine-tune: huấn luyện tiếp từ checkpoint

8.1 Fine-tune ở đâu?

Dùng script riêng `training/train_ecapa_finetune.py` — bản mở rộng của `train_ecapa.py`, thêm cờ `--resume-ckpt` để nạp checkpoint cũ rồi train tiếp. Chạy trên **Kaggle GPU** (T4) như khi train từ đầu.

8.2 Fine-tune như nào? (cơ chế)

Khác biệt duy nhất so với train thường là *nạp trọng số cũ trước khi vào vòng lặp*:

```
if args.resume_ckpt:
    ckpt = torch.load(args.resume_ckpt, map_location=device, weights_only=True)
    model.load_state_dict(ckpt["model"], strict=args.strict_resume) # nạp ECAPA cũ
    if args.resume_classifier and "classifier" in ckpt:
        classifier.load_state_dict(ckpt["classifier"], ...) # CHI khi cùng speaker set
# ... sau đó train bình thường: Adam + cosine LR + AAM-Softmax
```

Giải thích các cờ:

- `--resume-ckpt <path>`: file `best_model.pt` muốn fine-tune tiếp.
- `--strict-resume` (mặc định *tắt*): để `strict=False` cho phép nạp ngay cả khi có sai khác nhỏ về tham số — linh hoạt khi fine-tune.
- `--resume-classifier`: chỉ nạp lớp AAM-Softmax khi *cùng tập speaker/split*. Nếu đổi số người nói, hình dạng lớp khác → bỏ qua (script tự cảnh báo) và học classifier mới trên embedding cũ.
- `--save-every-epoch`: lưu `epoch_XXX.pt` mỗi epoch để Kaggle (giới hạn thời gian) có thể resume.

8.3 Khi nào nên fine-tune?

1. **Thêm augmentation**: lấy checkpoint sạch, fine-tune với `--augment --musan-root ... --rir-root ...` để tăng độ bền nhiều mà không mất kiến thức đã học.
2. **Thêm người nói mới**: enroll thêm user → fine-tune cập nhật model (thường *không* nạp classifier cũ vì số lớp đổi).
3. **Domain adaptation**: có checkpoint VoxCeleb, fine-tune trên giọng Việt để kéo về đúng miền (rẻ hơn train lại từ đầu).

```
# Fine-tune từ checkpoint sạch + bat augmentation
python train_ecapa_finetune.py \
    --data_root <data> --split_file <iden_split.txt> --save_dir results/ft \
    --resume-ckpt results/vivos/best_model.pt --resume-classifier \
    --epochs 10 --lr 5e-4 \
    --augment --specaugment --musan-root <musan> --rir-root <rir>
```

Mẹo: fine-tune thường dùng *learning rate nhỏ hơn* (vd 5×10^{-4}) và *ít epoch hơn* train từ đầu, để không “quên” kiến thức cũ.

9 Đánh giá mức cải thiện (before vs after)

Câu hỏi “fine-tune/augment có cải thiện không?” chỉ trả lời được khi so sánh **trên cùng một tập test/trial**. Quy trình chuẩn:

1. **Cố định tập đánh giá**: dùng *nguyên* `iden_split.txt` (test) và `veri_test.txt` (trial) của baseline — không sinh lại.
2. **Đo baseline**: chạy `evaluate_sid.py` + `evaluate_sv.py` trên checkpoint cũ → Top-1, EER, minDCF.
3. **Fine-tune** rồi đo lại trên *đúng* tập đó.
4. **So delta**: $\Delta\text{Top-1}$, ΔEER , ΔminDCF . EER/minDCF *giảm* = tốt; Top-1/F1 *tăng* = tốt.

Chỉ số	Baseline	Sau fine-tune	Δ (hướng tốt)
SID Top-1 (%)	...	...	↑
SID Macro-F1 (%)	...	...	↑
SV EER (%)	...	...	↓
SV minDCF	...	...	↓

Table 3: Mẫu bảng so sánh trước/sau fine-tune (điền số sau khi chạy thực nghiệm augment).

Cảnh báo khi kết luận “cải thiện”

Chênh lệch nhỏ trên trial set bé (vd VoxCeleb 552 cặp) có thể chỉ là *nhiều thống kê* chứ không phải cải thiện thật — xem variance V1 vs V3 (Mục 12.1). Muốn kết luận chắc, nên *bootstrap khoảng tin cậy* cho EER hoặc đánh giá trên trial set lớn (VIVOS 4,160 cặp).

Các chỉ số đánh giá — bảng tổng hợp

Chỉ số	Bài toán	Đo cái gì	Hướng tốt
Top-1 accuracy	SID	Tỉ lệ đoán đúng người ở hạng 1	cao
Top-5 accuracy	SID	Nhân đúng nằm trong 5 ứng viên đầu	cao
Micro-F1	SID	= Top-1 (single-label)	cao
Macro-F1	SID	Trung bình F1 theo từng người (nhảy lớp ít mẫu)	cao
EER	SV	Điểm FAR = FRR	thấp
minDCF	SV	Chi phí quyết định tối thiểu ($P_{tar}=0.01$)	thấp
Best val acc	Train	Accuracy validation cao nhất (chọn check-point)	cao

Table 4: Toàn bộ chỉ số dùng trong báo cáo và hướng “tốt”.

10 Đánh giá SID và F1-score

10.1 Suy luận SID bằng cosine với trọng số lớp

```
w_n = F.normalize(classifier.weight, dim=1)      # mỗi hàng = 1 centroid người nói
emb = F.normalize(model(mel).squeeze(1), dim=1)
logits = emb @ w_n.t()                          # cosine với từng người
top1 = logits.argmax(dim=1); top5 = logits.topk(5, dim=1).indices
```

Giải thích: mỗi hàng của ma trận trọng số AAM-Softmax đóng vai trò “centroid” một người. Dự đoán = người có cosine cao nhất. *Top-5* đo xem nhân đúng có nằm trong 5 ứng viên đầu.

10.2 F1-score: micro và macro

Với precision $P = \frac{TP}{TP+FP}$, recall $R = \frac{TP}{TP+FN}$: $F_1 = \frac{2PR}{P+R}$. Trong bài toán *single-label multiclass*:

$$\text{micro-}F_1 = \text{Top-1 accuracy}$$

(đẳng thức toán học: tổng TP = số mẫu đúng, $\text{micro-}P = \text{micro-}R = \text{accuracy}$). Vì vậy $\text{micro-}F_1$ điền được *ngay* từ kết quả hiện có. **Macro- F_1** là trung bình F_1 *từng người* (cần precision/recall theo lớp $\rightarrow$ confusion matrix).

10.3 Bổ sung tính macro-F1 (đã nâng cấp evaluate_sid.py)

```
cm = np.zeros((n_classes, n_classes), dtype=np.int64) # confusion matrix
for t, p in zip(y_true, y_pred): cm[t, p] += 1
tp = np.diag(cm); support = cm.sum(1); pred = cm.sum(0)
precision = tp / pred; recall = tp / support
f1 = 2*precision*recall / (precision+recall)
macro_f1 = f1[support > 0].mean() # trung bình trên các lớp có mat
weighted_f1 = (f1 * support).sum() / support.sum()
```

Trạng thái F1

Evidence ban đầu chỉ lưu Top-1/Top-5 nên **macro-F1 chưa có số**. Code trên đã sẵn sàng; dataset nằm trên Kaggle nên cần *một lần chạy lại* để điền số (Phụ lục 22). Micro-F1 = Top-1 đã đưa vào bảng kết quả.

11 Đánh giá SV: EER và minDCF

11.1 Trích embedding toàn utterance

```
wav = wav.mean(dim=0).unsqueeze(0) # mono, KHÔNG crop (dùng cả câu)
emb = F.normalize(model(feat(wav)).squeeze(1), dim=1)
```

Khác lúc train (crop 3s), SV trích embedding trên *toàn bộ* file để ổn định.

11.2 EER — Equal Error Rate

```
def compute_eer(labels, scores):
    fpr, tpr, thr = roc_curve(labels, scores); fnr = 1 - tpr
    idx = np.nanargmin(np.abs(fpr - fnr)) # điểm FAR == FRR
    return (fpr[idx] + fnr[idx]) / 2 * 100, thr[idx]
```

EER là điểm mà tỉ lệ chấp nhận sai (FAR) bằng tỉ lệ từ chối sai (FRR). *Càng thấp càng tốt*; threshold tại đó dùng để hiệu chỉnh ngưỡng runtime.

11.3 minDCF — chi phí quyết định tối thiểu

```
c_det = c_miss * fnr * p_target + c_fa * fpr * (1 - p_target) # p_target=0.01
c_def = min(c_miss * p_target, c_fa * (1 - p_target))
min_dcf = (c_det / c_def).min()
```

$$C_{\text{det}}(\tau) = C_{\text{miss}}P_{\text{miss}}(\tau)P_{\text{tar}} + C_{\text{fa}}P_{\text{fa}}(\tau)(1 - P_{\text{tar}}), \quad \text{minDCF} = \min_{\tau} \frac{C_{\text{det}}}{C_{\text{def}}}.$$

minDCF (theo NIST SRE, $P_{\text{tar}} = 0.01$) phạt nặng lỗi bỏ sót/báo nhầm trong kịch bản người mục tiêu *hiểm* — khắt khe hơn EER cho ứng dụng bảo mật.

11.4 Quyết định pass/block theo ngưỡng

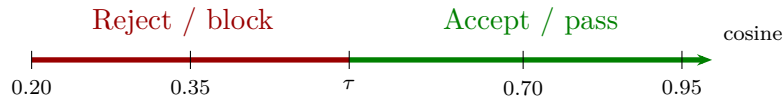


Figure 3: SV pass khi $\cos(x, c_{user}) \geq \tau$. Runtime dùng $\tau = 0.45$ cho ECAPA own checkpoint (xem core/config.py).

12 Kết quả và so sánh

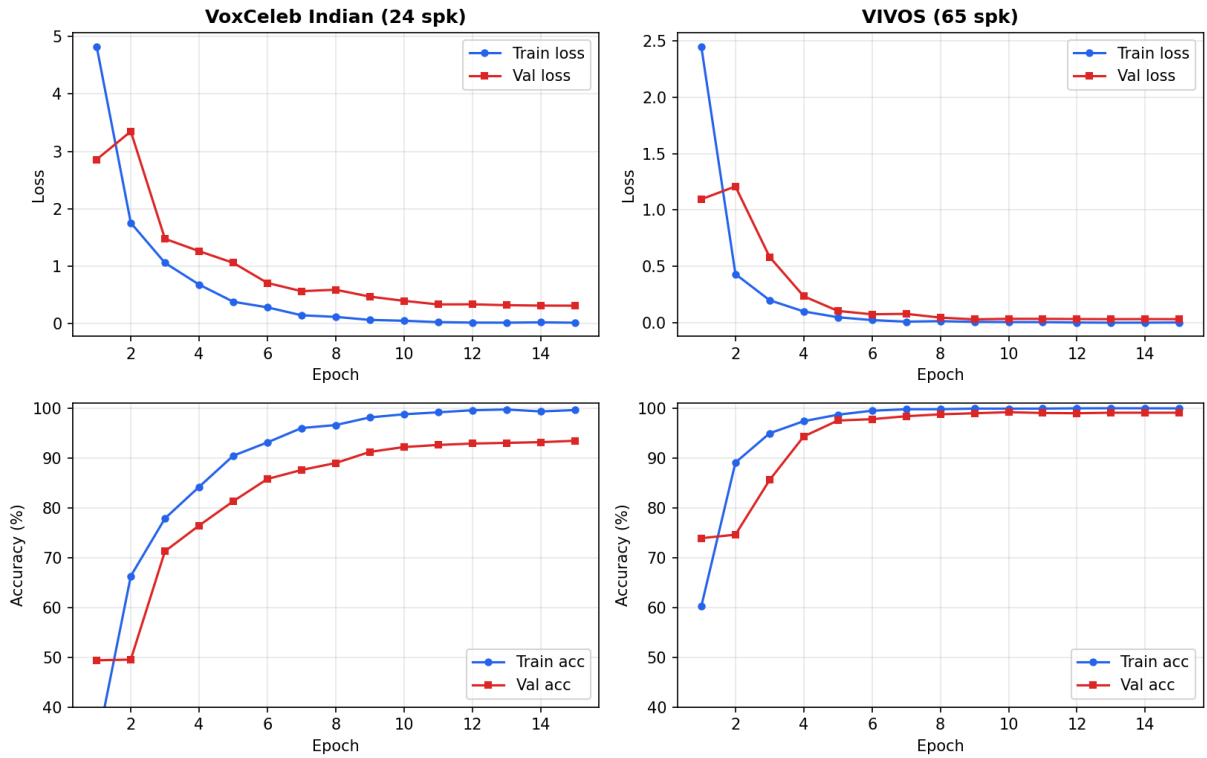


Figure 4: Loss (trên) và accuracy (dưới) theo epoch. Hội tụ ổn định, val loss không bật tăng (không overfit nặng).

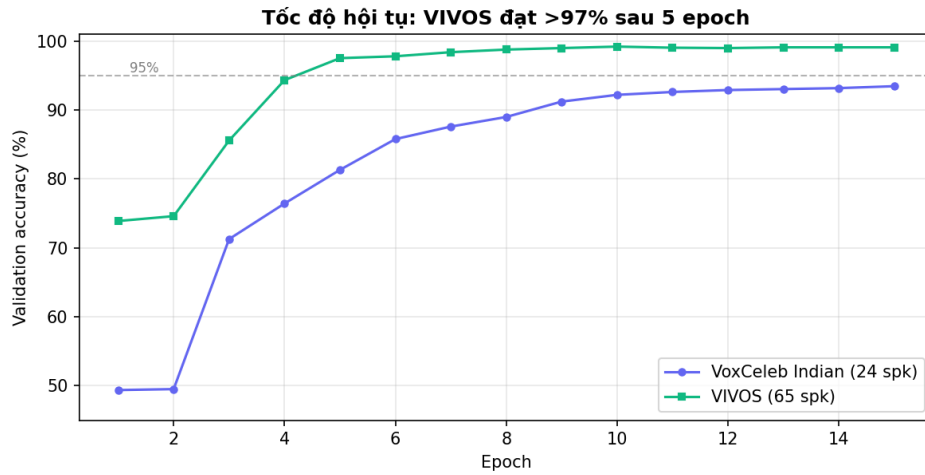


Figure 5: So sánh tốc độ hội tụ: VIVOS vượt 97% val-acc chỉ sau 5 epoch.

Chỉ số	VoxCeleb Indian	VIVOS
Best val accuracy	93.44%	99.19%
SID Top-1 / Micro-F1	91.34%	99.31%
SID Top-5	98.67%	100.00%
SID Macro-F1	<i>(cần chạy lại — Phụ lục 22)</i>	
SV EER	2.90%	0.96%
SV minDCF	0.1486	0.0673
Threshold @ EER	0.3884	0.4062
Số test / trial	751 / 552	1,883 / 4,160

Table 5: Bảng tổng hợp (từ sid_results.json, sv_results.json).

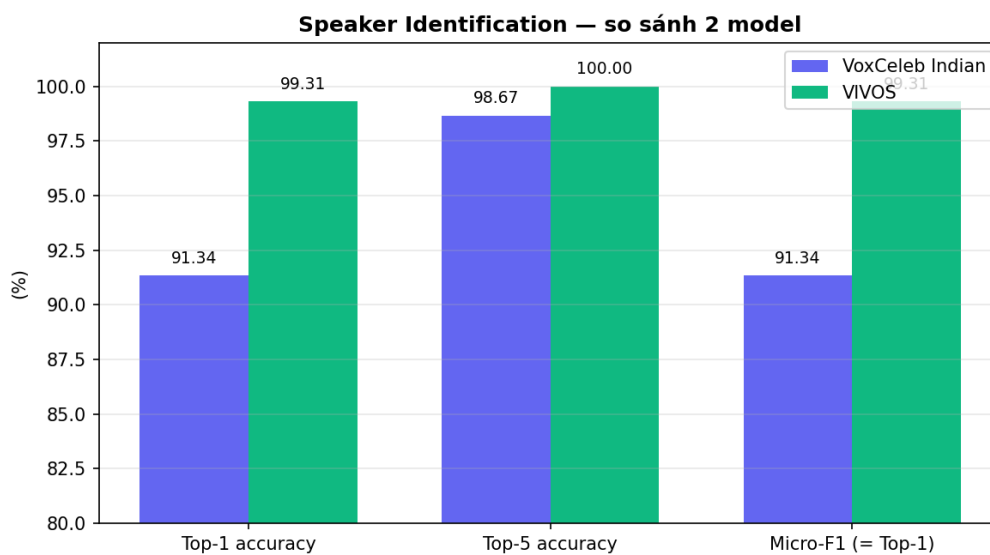


Figure 6: So sánh SID. VIVOS gần như hoàn hảo (Top-5 = 100%).

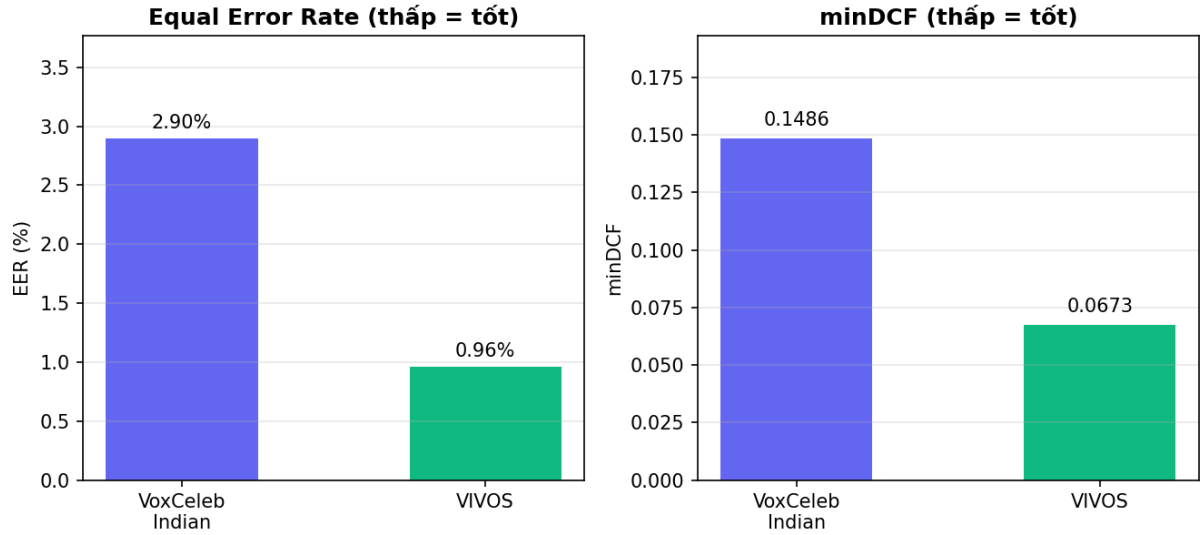


Figure 7: So sánh SV (thấp hơn = tốt hơn). VIVOS thấp hơn ở cả EER lẫn minDCF.

12.1 Lịch sử thực nghiệm và độ ổn định (V1/V2/V3)

Vòng	Dataset	Thiết lập	Kết quả
V1	VoxCeleb (24)	sơ bộ, 552 trial	Top-1 92.05%, EER 2.72%, minDCF 0.1014
V2	VIVOS (46, chỉ train/waves)	80/10/10 <i>không seed</i> , 20 epoch	Best val 99.56% (chưa eval)
V3	VoxCeleb (24) & VIVOS (65)	chính thức , seed=42, 15 epoch	Toàn bộ số ở bảng trên

Table 6: Ba vòng thực nghiệm; V3 sinh evidence trong **results/**.

Bảng chứng run-to-run variance

So V1 và V3 trên *cùng* VoxCeleb (khác lần xáo trộn): Top-1 92.05% $\rightarrow$ 91.34%, EER 2.72% $\rightarrow$ 2.90%. Mức dao động này **chứng minh bằng số** cho caveat “trial set 552 cặp có sai số thống kê”. VIVOS (4,160 trial) ổn định hơn nhiều.

13 Giải thích các file output

File	Nội dung & mục đích
best_model.pt	state_dict của ECAPA + lớp AAM-Softmax, kèm epoch, val_acc. File dùng để extract embedding lúc deploy.
spk2idx.json	Ánh xạ speaker ID → chỉ số lớp; bắt buộc giữ để giải mã dự đoán.
training_log.json	Loss/accuracy train+val theo epoch (vẽ đường cong).
hparams.json	Siêu tham số + môi trường (seed, Python, PyTorch, git SHA) — phục vụ tái lập.
sid_results.json	Top-1/Top-5 (+ micro/macro/weighted-F1 ở bản nâng cấp).
sv_results.json	EER, threshold, minDCF, số trial.
iden_split.txt	Split: <1 2 3> <đường_dẫn>.
veri_test.txt	Trial SV: <label> <file1> <file2>.
dataset_summary.json	Thống kê dataset (speaker, utterance, split, trial).
bad_audio_files.txt	(VIVOS) file lỗi đã loại: VIVOSSPK44/train_VIVOSSPK44_157.wav.

Table 7: Ý nghĩa từng artifact trong training/results/<dataset>/.

14 Lý do chọn mô hình và ưu điểm

Checkpoint chính cho trợ lý: bản huấn luyện trên VIVOS (checkpoints/vios/best_model.pt).

Làm rõ phạm vi “cải tiến”

Hai mô hình dùng cùng kiến trúc và siêu tham số; khác biệt là dữ liệu. Vì vậy ưu thế của checkpoint VIVOS so với checkpoint VoxCeleb đến từ phù hợp miền + chất lượng dữ liệu, không phải thay đổi mạng.

1. **Khớp miền tiếng Việt:** người dùng nói tiếng Việt; checkpoint VoxCeleb (giọng Ấn) là out-of-domain. VIVOS loại bỏ khoảng cách này.
2. **Chính xác hơn ở bài toán khó hơn:** 99.31% trên 65 người so với 91.34% trên 24 người.
3. **Xác minh tin cậy hơn:** EER 0.96% so với 2.90% ($\sim 3\times$ thấp hơn), minDCF 0.0673 so với 0.1486 → cổng Important an toàn hơn.
4. **Thống kê vững hơn:** 4,160 trial so với 552.
5. **Hội tụ nhanh, ổn định:** val-acc >97% sau 5 epoch.

VoxCeleb Indian vẫn được giữ để kiểm chứng tính tổng quát trên dữ liệu nhiều.

15 Hạn chế của phương pháp

1. Rò rỉ phiên (không group theo video_id); không phải VoxCeleb1-O.
2. Trial VoxCeleb nhỏ (552) → EER nhiều.

3. Lệch miền: VIVOS là giọng đọc sạch, dễ hơn thực tế.
4. Ngưỡng runtime chưa hiệu chỉnh trên người dùng thật.
5. Chưa bật augmentation trong evidence chính.
6. SV thuần cosine không chống được replay/voice-clone (cần challenge-response ở tăng ứng dụng).

16 Hướng cải tiến

1. **Chia split theo phiên:** nhóm theo `video_id`; dùng bộ trial chuẩn VoxCeleb1-O để so sánh với văn liệu.
2. **Bật augmentation** (MUSAN + RIR + SpecAugment + speed) — khung đã sẵn — để tăng độ bền với nhiễu, kỳ vọng giảm EER in-the-wild.
3. **Vẽ DET/ROC & score distribution:** dùng cờ `--scores-out` (vừa thêm vào `evaluate_sv.py`) để lưu (label, score), rồi plot đường cong.
4. **Confusion matrix & macro-F1:** chạy `evaluate_sid.py --confusion-out` (đã thêm), phân tích người nói bị nhầm nhiều nhất.
5. **AS-Norm:** chuẩn hoá điểm cosine theo cohort để ngưỡng ổn định giữa môi trường (runtime đã có AS-Norm-lite).
6. **Khoảng tin cậy EER:** bootstrap trên trial set, đặc biệt cho VoxCeleb.
7. **Tăng quy mô & fine-tune:** pre-train VoxCeleb1+2 rồi fine-tune trên tiếng Việt + giọng người dùng.
8. **Đo độ trễ & nén:** ghi nhận latency CPU/GPU, thử quantization để triển khai realtime.
9. **Anti-spoofing:** thêm mô hình phát hiện replay/synthetic.

17 Câu hỏi dự đoán và đáp án

Câu 1 — Vì sao chọn ECAPA-TDNN? Là baseline SOTA cho speaker embedding, kiểm chứng rộng rãi, có cài đặt SpeechBrain ổn định; SE-Res2Block + attentive pooling cho embedding tốt với chi phí hợp lý (~6.2M tham số).

Câu 2 — AAM-Softmax khác Cross-Entropy thường thế nào? Cộng biên góc $m = 0.2$ vào lớp đúng buộc embedding gần centroid hơn → inter-class lớn, intra-class nhỏ trong không gian cosine — đúng thứ SV/SID cần.

Câu 3 — Vì sao một mô hình dùng cho cả SID và SV? Train phân loại người nói tạo embedding phân biệt; SID = $\arg\max$ cosine, SV = ngưỡng cosine. Không cần hai mô hình riêng.

Câu 4 — Micro-F1 và Macro-F1 khác gì? Micro-F1 gộp toàn bộ mẫu → bằng accuracy ở bài single-label. Macro-F1 trung bình F1 theo *từng người*, nhạy với lớp ít mẫu/bị nhầm.

Câu 5 — EER là gì? minDCF là gì? EER là điểm FAR=FRR (càng thấp càng tốt). minDCF là chi phí quyết định tối thiểu khi người mục tiêu hiếm ($P_{tar} = 0.01$), khát khe hơn cho bảo mật.

Câu 6 — Vì sao VIVOS tốt hơn VoxCeleb dù nhiều người hơn? VIVOS là giọng đọc sạch, đúng miền tiếng Việt; VoxCeleb Indian là audio YouTube nhiều, đa kênh, lệch miền.

Câu 7 — Vì sao random crop khi train, full utterance khi verify? Random crop là augmentation thời gian, tăng tổng quát; verify cần embedding ổn định nên dùng cả câu.

Câu 8 — CMN để làm gì? Trừ trung bình theo thời gian để khử thiên lệch kênh/micro $\rightarrow$ bền hơn khi đổi thiết bị.

Câu 9 — Train acc gần 100% có overfit không? Val loss không bật tăng, val-acc cao (93–99%) và checkpoint chọn theo *best val* (early-best) nên overfit được kiểm soát; tuy vậy caveat rõ rĩ phiên khiến số liệu lạc quan hơn thực.

Câu 10 — Vì sao cần margin Top1–Top2 (SID) trước SV? Top-1 cao chưa chắc tin cậy nếu Top-2 sát; margin nhỏ = mơ hồ giữa hai người. Tầng router chặn trường hợp ambiguous trước khi xác minh.

18 Checklist ôn tập

#	Nội dung cần nhớ	OK
1	Một embedding ECAPA 192-d dùng cho cả SID (argmax cosine) và SV (ngưỡng cosine).	☐
2	Tiền xử lý: 16kHz $\rightarrow$ mono $\rightarrow$ crop 3s $\rightarrow$ mel 80 + log + CMN.	☐
3	AAM-Softmax ($m=0.2$, $s=30$) tạo biên góc $\rightarrow$ tách người nói tốt hơn CE.	☐
4	Adam + CosineAnnealingLR + grad clip 5.0; lưu checkpoint best-val.	☐
5	SID: Top-1/Top-5; micro-F1 = Top-1; macro-F1 cần confusion matrix.	☐
6	SV: EER (FAR=FRR), minDCF ($P_{tar}=0.01$); threshold runtime 0.45.	☐
7	VIVOS: 99.31% Top-1, EER 0.96% $\rightarrow$ checkpoint chính.	☐
8	VoxCeleb: 91.34%, EER 2.90%; trial nhỏ $\rightarrow$ EER nhiều (V1 vs V3).	☐
9	Caveat: rõ rĩ phiên, lệch miền, chưa augment.	☐
10	Cải tiến: group-split, augmentation, DET curve, macro-F1, AS-Norm.	☐

19 Tóm tắt một đoạn để nói

YC1 huấn luyện một mô hình ECAPA-TDNN sinh embedding 192 chiều phục vụ cả nhận diện (SID) lẫn xác minh người nói (SV). Audio được đưa về 16kHz mono, cắt 3 giây, trích mel-filterbank 80 chiều rồi log + CMN; mạng được train bằng AAM-Softmax (biên góc 0.2, scale 30) với Adam + cosine LR + gradient clipping, lưu checkpoint theo val accuracy cao nhất. SID đo Top-1/Top-5 (micro-F1 = Top-1), SV đo EER và minDCF bằng cosine similarity giữa các embedding. Trên VIVOS, mô hình đạt Top-1 99.31% và EER 0.96%, vượt trội checkpoint VoxCeleb Indian (91.34%, 2.90%) chủ yếu nhờ khớp miền tiếng Việt và dữ liệu sạch — nên VIVOS được chọn làm checkpoint chính. Các hạn chế (rõ rĩ phiên, trial nhỏ, chưa augment) và hướng cải tiến (group-split, augmentation, DET curve, macro-F1, AS-Norm) đều được nêu rõ.

20 Gợi ý thứ tự trình bày 5 phút

1. **30s** — Bài toán: SID vs SV, vì sao một embedding dùng cho cả hai.
2. **60s** — Tiền xử lý + mel + CMN (sơ đồ pipeline).
3. **60s** — ECAPA-TDNN + AAM-Softmax (vì sao biên góc).
4. **60s** — Đánh giá: Top-1/F1, EER, minDCF (đường cong + bảng).
5. **60s** — So sánh VoxCeleb vs VIVOS, lý do chọn VIVOS.
6. **30s** — Caveat + hướng cải tiến.

21 Kết luận

Phần YC1 xây dựng lõi sinh trắc giọng nói: một mô hình ECAPA-TDNN, train bằng AAM-Softmax, dùng cosine similarity cho cả SID và SV. Quy trình từ tiền xử lý, kiến trúc, huấn luyện đến đánh giá đều được kiểm soát và tái lập (seed, hparams), với evidence thật trên hai dataset. Checkpoint VIVOS được chọn nhờ khớp miền tiếng Việt và chỉ số tốt nhất. Báo cáo trình bày trung thực cả điểm mạnh lẫn hạn chế, và để sẵn các móc nối kỹ thuật (cờ `--scores-out`, `--confusion-out`, khung augmentation) cho các cải tiến tiếp theo.

22 Phụ lục: lệnh tái lập

```
# Train
python train_ecapa.py --data_root <data> --split_file <iden_split.txt> \
  --save_dir results/<dataset> --epochs 15 --batch_size 64 --lr 1e-3

# Đánh giá SID (đã có macro-F1 + confusion matrix)
python evaluate_sid.py --ckpt results/<dataset>/best_model.pt \
  --spk2idx results/<dataset>/spk2idx.json \
  --data_root <data> --split_file <iden_split.txt> \
  --out results/<dataset>/sid_results.json \
  --confusion-out results/<dataset>/confusion.json

# Đánh giá SV (lưu thêm scores để vẽ DET/ROC)
python evaluate_sv.py --ckpt results/<dataset>/best_model.pt \
  --data_root <data> --trial_file <veri_test.txt> \
  --out results/<dataset>/sv_results.json \
  --scores-out results/<dataset>/sv_scores.json
```

Nguồn: `training/train_ecapa.py`, `evaluate_sid.py`, `evaluate_sv.py`, `augmentations.py`, `notebooks/SecureVA-V3.ipynb` và `training/results/`. Biểu đồ tạo bằng `training/report/make_figures.py`.
Kiến trúc: Desplanques et al., *ECAPA-TDNN*, Interspeech 2020; SpeechBrain.